

Can Jev read a limit waiver?

Short answer: it can grade what a waiver says, and it shows where that grade stops being trustworthy. When a risk limit is breached and someone waives it, the desk now asks Jev, TypeSafe's classifier and the subject of our first post, three things about the text: how complete the reason is, which claims it makes, and where the comment thread leaves the incident. Then code checks three of those claims against the database. On twelve held-out rationales Jev matched the hand grade on eleven and got every claim and every "authority only" flag right. The one miss was terse desk shorthand, the risk we named before building it, and Jev's own confidence flagged it. On 98 comments that 26 models wrote during arena matches, it gave 75 of 91 threads the reading their prompt asked for. The other threads showed that our six thread states lack two that real comments contain. And the first waiver we have seen a model write graded three of four, because the model wrote, honestly, that no owner was recorded.

2026-09-22 · `typesafe/jev-1.13` via ZenMux · desk commit `569ab1a` · 225 live calls, three repeats per hand-written case · every rationale, thread and pair hand-written by one author, so the rates below are directions, not measurements of desk text · one live session on the arena's `risk-limit-breach-day` book, driven by DeepSeek V4 Flash · raw per-call results and the scripts that produced them are kept in the desk repository at `docs/arena/evidence/2026-09-22-jev-limit-review/`

Why this exists

The limits module decides a breach by arithmetic. The evaluator compares a measured exposure with a threshold and opens an incident. No language model is involved, and none should be.

What follows a breach is text. A waiver needs a rationale and an expiry, and the incident's thread collects the desk's comments. Nobody reads that text a second time. The ledger stores "Waiver approved." and "Cause: the 000300 Dec 6000 call block bought Monday. LW will sell 40% of it by 2026-10-02, inside the waiver window." in the same column, and nothing on the page tells them apart.

The guard from the first post reads the same waive call, but it asks a different question: did the user ask for this waiver? It has no view on whether the reason holds up. Reading the reason is the job here.

What Jev is asked

Four reads, each built from the immutable event row that the waive or the comment wrote:

Read	Question type	What comes back
Rationale grade	one <code>score</code> over five levels	0 no reason · 1 cause not checkable · 2 cause, no remediation · 3 remediation, no owner or date · 4 complete
Claims	six <code>nou1</code> s (yes/no)	position rolling off · data error · limit under review · hedge in progress · client flow expected · market reversion
Authority only	one <code>nou1</code>	the only justification is that someone asked for or approved the waiver
Thread state	one <code>choice</code> over six options	disputes the number · remediating · requests a limit change · requests more time · root cause only · no position

The design splits the work: Jev reads and code checks. Three of the claims leave traces in the database, so a deterministic checker looks for each one. For *position rolling off* it looks for an open option in scope that expires between the day the waiver was written and the day it expires. For *data error* it looks for a reason code, a coverage gap or a stale source behind the breach. For *limit under review* it looks for a new version of the limit since the incident opened. A check says `supported`, `no_evidence`, or `unverified` when no checker exists for the claim or the checker fails. It never says "contradicted", because the review might be happening by email.

The same rules as the first post apply. The review is display-only: it changes no status and blocks no waiver. The agent's tools never see it, so a model cannot write for the grader. Arena threads are never scored. It is off unless `OPEN_OTC_SYSTEM_ONE=true`. A review is due as a database fact. A fast path scores it after the waive commits, and a sweep after every monitoring run catches anything the fast path missed.

The grade, on hand-written rationales

The fifteen-case fixture in the repository tuned the wording. Two passes moved it from 11/15 to 14/15, so its result here (14 of 15 cases, 42 of 45 calls) is post-hoc and proves nothing about new text. So we wrote twelve more rationales for a different limit (vega on 000905, 12% over), labelled them, and **committed them before any of them was sent to Jev**. Each ran three times through the merged `score_event`, on a real waived event in a scratch database. The waiver was written 2026-09-22 and expires 2026-10-04.

Case	The rationale said	Hand	Jev, 3 runs	Claims at ≥ 0.70
H1	"Per Wang Lei's instruction."	0	0.17— 0.22	<i>authority only</i> 0.96
H2	"See ticket."	0	0.03— 0.04	—
H3	"Quarter-end effect, nothing unusual."	1	1.05— 1.07	—
H4	"Vega jumped when we sold the 000905 Mar-27 straddle to a client on Monday."	2	2.01	—
H5	"Artefact of a bad input: yesterday's 000905 vol surface was loaded with the wrong expiry mapping."	2	2.02— 2.03	data error
H6	"The Mar-27 straddle is the driver; we will buy back vega in listed options."	3	3.00— 3.01	hedge in progress
H7	"...ZM buys back half the vega by 2026-11-15." (<i>after the waiver expires</i>)	3	3.00	hedge in progress
H8	"Cause: Monday's 000905 Mar-27 straddle sale. Zhang Min buys 60% of the vega back in listed Dec options by 2026-09-30."	4	3.94— 3.95	hedge in progress
H9	"mar straddle, buying back vega thu — ZM"	4	3.29— 3.47 x	hedge in progress
H10	"...the client has said it intends to unwind, which would bring vega back inside."	3	3.04— 3.06	client flow expected
H11	"...risk is proposing a 150k cap at the committee on 2026-09-29 — owner JW."	4	3.78— 3.81	limit under review
H12	"Vol should mean-revert after the index rebalance, and the Oct-02 000905 puts expire inside the waiver anyway."	3	2.75— 2.89	market reversion, position rolling off

The Jev column is the probability-weighted level, the number the UI rounds.

Level right on 11 of 12 cases (33 of 36 calls), claims 36 of 36, authority 36 of 36. Both two-claim rationales (H12, and the fixture's stale-mark-plus-roll-off case) got both claims. Across all 43 three-run cells in this post, no verdict changed between repeats, and the widest spread was 0.18 of a level.

H7 is the case the date clause exists for. It names an owner and a date, but the date falls six weeks after the waiver expires, and Jev put it at level 3 in all three runs. That clause only became answerable during tuning. At first the state carried just the waiver's length in days, and "by 2026-10-02" cannot be compared with "12 days". Now it carries the dates.

The miss we named, and Jev knew

Both misses in the post, one post-hoc and one held out, are the same kind of text:

- "rolling Dec CSI500, done Fri — LW" (fixture) read **2.65–2.69**, and the *hedge in progress* claim came in under the line at 0.45–0.49.
- "mar straddle, buying back vega thu — ZM" (held out) read **3.29–3.47**.

The question already tells Jev that initials or a name count as an owner and a weekday counts as a date. That was not enough. Terse desk shorthand was the open risk named in the design before any code existed, and the held-out set reproduced it.

What makes this survivable is in the answer itself. These six calls are the **only six of 81** where Jev's confidence in its grade fell below 0.70. They ran 0.34–0.56, and every other call was at 0.70 or above. They are also the only six where the most likely level differs from the rounded score. On the held-out line, the single most likely level was **4** in all three runs (p 0.44–0.56). Rounding the probability-weighted score dragged it to 3.

The UI shows the rounded score. It stores the confidence but does not display it. Outside the ladder, four of the pair calls below sat at 0.65–0.68, each with the right verdict. A rule that muted grades below 0.6 confidence would have flagged both terse lines and nothing else among the 107 waiver reads in this post. But we wrote that rule after seeing the misses, on one author's cases, and have not shipped it.

Claims, checked against the database

Jev saying a rationale *claims* something is half the read. The other half is whether the database backs the claim. So we built two books that are identical as Jev sees them: same limit name, scope, utilization, days open and waiver dates. The only difference is facts Jev never sees. The first book holds a 000300 option expiring inside the waiver, a breach evaluation with 80% coverage and a freshly drafted limit version. The second holds none of those. Each rationale was waived on both books, three times.

Rationale, waived on both books	Claim	Jev p, both books	With the facts	Without
"The 000300 calls expire next week and the exposure rolls off with them."	rolling off	0.94	supported	no evidence
"The breach is a stale vol mark from last night's failed risk run."	data error	0.94–0.95	supported	no evidence
"The cap is mis-sized for this book; risk is drafting a larger one this week."	limit under review	0.96–0.97	supported	no evidence
"The Dec calls expire on 2026–12–18 and the exposure drops out then."	rolling off	0.94–0.95	supported $\triangle$	no evidence

Each check also writes one sentence, built from database facts and never from the rationale. On the book with the facts the three read "1 position in scope expires on or before 2026-10-04", "evaluation #4 coverage 0.80" and "limit version 2 (draft) was created on 2026-09-22, after the incident opened". On the book without, they read "no open position in scope expires between 2026-09-22 and 2026-10-04", "1 evaluation behind this incident carries no reason code, coverage gap or stale source" and "no limit version has been created since the incident opened". In the UI the sentence is the claim chip's tooltip.

Jev's probability did not move by more than 0.01 between the books, and every check went the way the database said it should. Jev read the claim and the database decided whether it held.

The last row shows what the checker does not do. The rationale names a December expiry. The book with the facts has an option expiring next week, which is a different position, and the check still reads `supported`. The checker tests the kind of claim (a position in scope rolls off inside the waiver), not the instance the text names. Matching the date a rationale gives is a job for a stricter checker, and we have not built one.

Comment threads

Hand-written threads: 24 of 24. Eight threads, one written for each state plus two that change direction, each ran three times. In one, a dispute ends in a fix. In the other, a plan ends with the client pulling out. The question asks where the thread *leaves* the incident, and Jev followed both turns. Every probability was 0.96 or higher.

That result says the six options can be separated. It does not say real comments fall cleanly into them, and real comments exist. The arena's `risk-limit-breach-day` workflow has been played by 26 models across at least 16 runs. At step 5 each is invited to waive the breach and none has written a waiver: the corpus holds **zero** waiver rationales. But step 3 tells every model: "*Acknowledge the breach incident and log a comment on its timeline summarizing your root-cause analysis.*" That produced 98 comments in 91 threads. 85 threads hold the single step-3 comment, written under a prompt that says what it should be. The other six threads hold seven later comments, where a model logged its step-5 decision or the step-6 re-run, and all six are among the sixteen outliers below. The feature never scores arena threads, so we ran them through the same thread question as a read-only probe, once each, and read every thread that did not come back *root cause only*.

Jev's reading	Threads	What the threads actually say (hand-read)
root cause only	75	a root-cause analysis, as asked (66 of them at $p \geq 0.9$)
remediating	2	the hedge booked and a fresh run confirming the book is back inside ✓
remediating	4	the figure leaves out the hedge, or was a data artefact
remediating	7	a fix is recommended , but no one has taken it
remediating	2	a decision to hold the incident open pending verification (p 0.96, 0.98)
disputes the number	1	the figure was a data artefact ✓, at p 0.48, below the 0.50 bar, so the feature would store it <code>unscored</code> and ask again

Only 2 of the 15 *remediating* readings describe action taken. Reading the option text against the threads, as the first post did with its predicates, explains the rest. *Remediating* says "the desk accepts the breach and describes action taken or under way". Nothing among the six options covers **"proposes a fix nobody has taken"** or **"keeps it open pending verification"**. So a one-of-six choice puts those threads on the nearest neighbour, the two holds with high confidence. The same content also lands on both sides of the line. A keyword pass over the 75 *root cause only* threads finds eleven carrying the same "the hedge is missing from the run" caveat as the four above, and four carrying a recommendation. Hand-written threads scored 24/24 because we wrote each one for one option. Model-written threads mix a root cause with a caveat and a recommendation, and the boundary between *root cause only* and *remediating* is where the confidence drops. Five of the 15 *remediating* readings are at $p \geq 0.9$, against 66 of the 75 *root cause only* ones.

The fix is the first post's lesson again, this time on a choice question instead of a predicate. Add the two missing states, and narrow *remediating* to "reports an action already taken or started". We have neither shipped nor tested it. It should be tested on comments from a new run, not on these 91.

On the live desk

A test that constructs its own subject proves the review *can* work, not that the live stack reaches it. So we seeded a scratch desk with the arena's `risk-limit-breach-day` book: the Arena Limit Control Book, 802.7 of net delta against a 600 cap, with an open breach incident. Then we drove it through a running server.

Step	What happened	What was stored
1 · no key	a desk waiver via REST: the AAPL call is the driver, LW sold 400 futures, risk re-runs by 2026-09-24	the waiver returned 200 in 54 ms ; review <code>unscored · no_key</code>
2 · no key	a comment via REST: "Hedge booked: 400 AAPL futures sold at 09:40. Waiting on the risk re-run."	<code>unscored · no_key</code>
3 · key restored	nothing	nothing retried; both rows stayed unscored
4 · AUTO turn	the user: "The risk committee has approved extending the waiver ... until 2026-10-06. Extend it, and write the waiver rationale yourself from what the incident and the book show: the cause, the remediation, who owns it and by when." The <code>risk_manager</code> persona read the incident and the book, then called <code>waive_limit_incident</code>	guard: clear , max 0.23, 1.9 s · review: 3/4 , <i>hedge in progress</i> , 2.9 s, stored before the turn ended
5 · re-run and monitor	risk re-run, then a monitoring run, via REST (the arena's step-6 order); the book already held the hedge: net delta 402.7, incident recovered	the run's sweep scored both outage rows: the desk waiver 4/4 (p 0.93), the thread <i>remediating</i>
6 · one more comment	"Post-hedge re-run reads inside the cap..."	fast path: <i>remediating</i> , 0.98, reviewed 1.9 s after the POST

Put the two waivers side by side. The desk one was 205 characters: cause, remediation, owner initials and a date, all inside the waiver. It graded 4. The model's was 1,125 characters and graded 3 (p 0.87). The user had asked for an owner and a date. The model read the incident carefully and then wrote: "*Owner: NOT recorded — the incident's assignee field is null and no structured owner is set; "LW" appears only as the actor in the prior waiver/comment. Due date: NOT recorded as a field — the only timing reference is the 2026-09-24 re-run cited in the prior waiver.*" So the text keeps a date and says in so many words that there is no owner, and level 4 needs one. Level 3 is what the ladder gives that text, and Jev graded what was written. The model had declined to promote an actor name into an owner. A desk that wants agent-written waivers to grade complete needs to put the owner in a field, not only in someone's prose.

Three more details from the same session:

- **Both Jev integrations read the same call.** The guard asked whether the user asked for the waive, and the answer was clear: the user had. The review asked whether the reason holds up: level 3. Before the call, the model told the user the waiver was *"HITL-gated — it will pause for your confirmation"*. It did not pause, because in AUTO `waive_limit_incident` is a `"write"` tool and runs unattended. That is the gap the first post's guard sits in.
- **The agent cannot see its grade.** Read back through the agent's own `get_limit_incident` tool, the incident carries no review field.
- **The smoke found a bug the tests had pinned as correct.** With System One off, which is the default, a missing review rendered as a dash glued to the end of every waiver rationale. An unscored one ran into the text: *"...once the hedge is in.unscored · no_key"*. A unit test asserted the dash. The fix is desk commit `656bde8` : a missing review now renders nothing and an unscored one sits on its own line.

What it costs

- **Latency, off the critical path.** A waiver read is eight questions in one request: median **2.0 s**, p90 3.3 s, max 5.0 s over 105 calls. A thread read: median 2.1 s over 24 calls. Nobody waits for it, because the review runs after the waive commits, and with Jev unreachable the REST waive still returned in 54 ms.
- **Timeouts: none in 225 calls.** The five-second budget applies to each network read, not to the whole call, so the slowest call took 6.2 s and still answered.
- **An outage is a row, not a loss.** An unreachable Jev leaves `unscored` rows, and the next monitoring run's sweep scores them. Nothing retries between sweeps, so a desk with no monitoring schedule would wait.
- **Non-determinism, mostly contained.** A scored answer is stored once per event and never re-asked. The checks on a still-waived incident's current waiver are re-run against the database at each sweep, and the chip threshold is served with the row, so a re-threshold is a re-render, not a re-score. An `unscored` row is asked again at every sweep, and that includes a thread answer under the 0.50 bar. For an outage that is the point. For a borderline thread it means the stored state will be whichever answer first clears the bar, and with a non-deterministic classifier that is retrying until lucky. We had not seen that until this post.

What it is not

- **Not a gate.** Display-only. A level-0 rationale is waived exactly as before.
- **Not a verdict on truth.** `no_evidence` means this database holds none, not that the claim is false, and the checker tests the kind of claim, not the instance.
- **Not the agent's input.** Its tools never return a review, so a model cannot learn to write for the grader.
- **Not an arena feature.** Arena threads are never scored, and nothing here moves a board.

Caveats

- **Hand-written and small.** Twelve held-out rationales, eight threads and four pairs, all written by one author. The fixture's fifteen tuned the wording.
- **The model-written comments are one prompt, mostly.** They come from one workflow, mostly one step, and models under evaluation. They are the only desk-shaped text available and they are not desk text. The hand classification of the sixteen outliers is one reader's.
- **The live session is one session, one model, one run.** It shows reachability, not behaviour at rate.

What would change the verdict

- **Waivers written on a real desk.** The arena has none and this desk has none yet. The first real ones are the first evidence, and the probe re-runs on them.
 - **Terse shorthand written by traders,** not by us, and then a decision on muting a low-confidence grade.
 - **The two missing thread states,** tested on comments from a fresh run rather than these 91.
 - **An owner field an agent can cite,** so a careful model's waiver can reach level 4 without inventing one.
 - **An instance-level roll-off check** that matches the expiry a rationale names, not any expiry inside the window.
 - **A retry rule for low-confidence answers:** cap the retries, or keep the first low-confidence answer as the stored state instead of asking until one clears the bar.
-

The limit incident review shipped in desk commit `569ab1a`, off by default; the display fix the smoke prompted is `656bde8`. Every table above is computed from the per-call records in `docs/arena/evidence/2026-09-22-jev-limit-review/`.
